

12-位运算

一、本阶段目标

位运算是 C++ 中直接对二进制位进行操作的工具。

这一章的目标是：

- 理解整数在计算机中可以看成二进制
- 掌握 `&`、`|`、`^`、`~`、`<<`、`>>`
- 会用位运算判断奇偶
- 会用位运算乘除 2
- 理解异或的常见性质
- 初步认识状态压缩思想

位运算的核心是：

```
1  把一个整数看成一串 0 和 1，然后直接操作这些二进制位。
```

二、二进制基础

十进制：

```
1  13
```

二进制表示：

```
1  1101
```

因为：

```
1  13 = 8 + 4 + 1
```

也就是：

```
1  13 = 2^3 + 2^2 + 2^0
```

三、按位与 &

1. 含义

& 表示按位与。

规则：

- 1 两个位置都是 1，结果才是 1。
- 2 否则是 0。

2. 示例

```
1      1101
2  & 1011
3  -----
4      1001
```

3. 常见用途：判断奇偶

```
1  if(x&1)
2  {
3      cout<<"奇数";
4  }
5  else
6  {
7      cout<<"偶数";
8  }
```

原因：

- 1 二进制最后一位是 1，说明是奇数；
- 2 二进制最后一位是 0，说明是偶数。

四、按位或 |

1. 含义

| 表示按位或。

规则：

- 1 两个位置只要有一个是 1，结果就是 1。

2. 示例

```
1      1101
2      | 1011
3      -----
4      1111
```

3. 常见用途

常用于：

- 1 把某一位设置成 1。

例如：

```
1  x=x|(1<<k);
```

表示：

- 1 把 x 的第 k 位变成 1。

五、按位异或 ^

1. 含义

^ 表示按位异或。

规则：

- 1 两个位置不同，结果是 1；
- 2 两个位置相同，结果是 0。

2. 示例

```
1      1101
2  ^   1011
3  -----
4      0110
```

3. 异或的重要性质

```
1  x ^ x = 0
2  x ^ 0 = x
```

4. 常见用途

如果一组数中，除了一个数出现一次，其他数都出现两次，可以用异或找出只出现一次的数。

```
1  int ans=0;
2
3  for(int i=0;i<n;i++)
4  {
5      ans=ans^a[i];
6  }
```

因为：

```
1  相同的数异或会抵消成 0。
```

六、按位取反 ~

1. 含义

~ 表示按位取反。

规则：

```
1  0 变 1
2  1 变 0
```

2. 注意

在 C++ 中，整数通常使用补码存储。

所以：

```
1  ~x
```

结果不只是简单地看表面二进制，初学阶段不建议大量使用。

现阶段只需要知道：

```
1  ~ 是把每一位取反。
```

七、左移 <<

1. 含义

```
1  x<<k
```

表示：

```
1  把 x 的二进制整体向左移动 k 位。
```

2. 效果

通常相当于：

```
1  x × 2^k
```

3. 示例

```
1  int x=3;
2
3  cout<<(x<<1);
```

输出：

```
1  6
```

因为：

```
1  3 的二进制是 11
2  左移一位变成 110
3  也就是 6
```

4. 常见写法

```
1  1<<k
```

表示：

```
1  2^k
```

例如：

```
1  1<<3
```

结果是：

```
1  8
```

八、右移 >>

1. 含义

```
1  x>>k
```

表示：

```
1  把 x 的二进制整体向右移动 k 位。
```

2. 效果

对于非负整数，通常相当于：

```
1  x / 2^k
```

向下取整。

3. 示例

```
1  int x=13;
2
3  cout<<(x>>1);
```

输出：

```
1  6
```

因为：

```
1  13 / 2 = 6
```

九、常用位运算操作

1. 判断第 k 位是不是 1

```
1  if(x&(1<<k))
2  {
3      cout<<"第 k 位是 1";
4  }
```

2. 把第 k 位变成 1

```
1 x=x|(1<<k);
```

3. 把第 k 位变成 0

```
1 x=x&(~(1<<k));
```

4. 反转第 k 位

```
1 x=x^(1<<k);
```

如果原来是 1，会变成 0。

如果原来是 0，会变成 1。

十、位运算和集合状态

1. 一个整数可以表示一个集合

例如有 4 个物品：

```
1 0 1 2 3
```

可以用二进制表示选没选。

例如：

```
1 1011
```

表示：

```
1 选了 0、1、3
2 没选 2
```

2. 为什么？

从右往左看：

```
1 第 0 位：1，选了 0
2 第 1 位：1，选了 1
3 第 2 位：0，没选 2
4 第 3 位：1，选了 3
```

3. 枚举所有状态

如果有 `n` 个物品，每个物品有选或不选两种状态。

状态总数：

```
1 2^n
```

可以写成：

```
1 for(int s=0;s<(1<<n);s++)
2 {
3     // s 表示一种选择方案
4 }
```

4. 判断某个物品是否被选

```
1 if(s&(1<<i))
2 {
3     // 第 i 个物品被选了
4 }
```

十一、位运算常见用途

操作	写法	
判断奇偶	<code>x&1</code>	
乘以 2	<code>x<<1</code>	
除以 2	<code>x>>1</code>	

操作	写法	
计算 2^k	<code>1<<k</code>	
判断第 k 位	<code>x&(1<<k)</code>	
设置第 k 位为 1	<code>`x</code>	<code>(1<<k)`</code>
设置第 k 位为 0	<code>x&(~(1<<k))</code>	
反转第 k 位	<code>x^(1<<k)</code>	
枚举所有子集	<code>for(int s=0;s<(1<<n);s++)</code>	

十二、常见错误

1. 忘记括号

错误：

```
1  if(x&1==1)
```

由于运算优先级问题，可能不是想要的效果。

建议写：

```
1  if((x&1)==1)
```

或者简单写：

```
1  if(x&1)
```

2. $1<<k$ 溢出

如果 k 很大， $1<<k$ 可能超出 `int` 范围。

可以写：

```
1  1LL<<k
```

3. 位数从 0 开始

第 0 位是最低位。

例如：

```
1  二进制 1010
```

从右往左：

```
1  第 0 位是 0
2  第 1 位是 1
3  第 2 位是 0
4  第 3 位是 1
```

4. 不要把 ^ 当成幂

C++ 里：

```
1  2^3
```

不是 8。

它是异或。

如果要表示 2^3 ，可以写：

```
1  1<<3
```

或者用：

```
1  pow(2,3)
```

但算法题里整数幂常常优先用位移或循环。

十三、本专题做过的题

题号 / 题目	类型	题面简述	对应知识点
P1149 火柴棒等式	枚举 / 位思维前置	用火柴棒拼数字，枚举满足条件的等式	枚举、状态意识
P2397 yyy loves Maths VI	异或	找出只出现奇数次或特殊出现次数的数	异或抵消
状态压缩入门题	位运算	用一个整数表示多个开关状态	子集枚举

十四、本章最重要的结论

位运算的核心是：

- 1 把整数当成二进制状态来操作。

最常用的三个能力是：

- 1 判断某一位
- 2 修改某一位
- 3 枚举所有状态

现阶段重点掌握：

- 1 $\&$ 判断
- 2 $|$ 设置
- 3 $\wedge$ 反转 / 抵消
- 4 $\ll$ 生成 2 的幂

后面学习状态压缩 DP 时，位运算会再次出现。